

BANKER'S ALGORITHM

R.PRABHAKARA ARJUN

2022503003

WHAT IS A DEADLOCK?

- A deadlock is a situation in which two or more process are each waiting for the other to finish, preventing any of them from progressing.
- In other words, it's a state in which each member of a group is waiting for another member to take action that can't be completed because everyone is waiting for someone else to act.

WHAT IS BANKER'S ALGORITHM?

- The Banker's algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems.
- It was developed by Edsger Dijkstra in 1965 and is designed to prevent deadlock in a system by ensuring that the system will be in a safe state before allocating resources to processes.

WHAT IS SAFE STATE?

- In a safe state, the system can ensure that all processes will eventually complete their execution without getting stuck due to resource conflicts.
- For a system to be in a safe state, it must satisfy the following conditions:
 1. **Mutual Exclusion:** Each resource must be either currently allocated to exactly one process or available for allocation.
 2. **Hold and Wait (Resource Holding):** Processes currently holding resources can request additional resources. However, they must release the held resources if they cannot immediately acquire the additional resources they need.
 3. **No Preemption:** Resources cannot be forcibly taken away from a process; a process can only voluntarily release resources it holds.
 4. **Circular Wait:** There must not exist a circular chain of processes, each of which is waiting for a resource held by the next process in the chain. In other words, there should be no circular dependency of resource requests among processes.

PREREQUISITE

- **Available:** A vector of length m indicates the number of available resources of each type. If $\text{Available}[j]$ equals k , then k instances of resource type R_j are available.
- **Max:** An $n \times m$ matrix defines the maximum demand of each process. If $\text{Max}[i][j]$ equals k , then process P_i may request at most k instances of resource type R_j .
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i][j]$ equals k , then process P_i is currently allocated k instances of resource type R_j .
- **Need:** An $n \times m$ matrix indicates the remaining resource need of each process. If $\text{Need}[i][j]$ equals k , then process P_i may need k more instances of resource type R_j to complete its task. Note that $\text{Need}[i][j]$ equals $\text{Max}[i][j] - \text{Allocation}[i][j]$.

SAFETY ALGORITHM:

- Let Work and Finish be vectors of length m and n, respectively.
Initialize Work = Available and Finish[i] = false for $i = 0, 1, \dots, n - 1$.
- 2. Find an index i such that both
 - a. Finish[i] == false
 - b. $Need_i \leq Work$,If no such i exists, go to step 4.
- 3. $Work = Work + Allocation_i$ Finish[i] = true Go to step 2.
- 4. If Finish[i] == true for all i, then the system is in a safe state,
If Finish[i] == false for all i, then the system is in unsafe state.

PROBLEM-1

IS THE SYSTEM IN SAFE STATE OR NOT?

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3				3	3	2
P1	2	0	0	3	2	2						
P2	3	0	2	9	0	2						
P3	2	1	1	2	2	2						
P4	0	0	2	4	3	3						

STEP-1:finding the need.

$$\text{Need}_i = \text{Max}_i - \text{Allocation}_i$$

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

STEP-2: Finding the values for available_i

- Check which need is satisfied by the available resource.
- Allocate that resource. i.e) $available_i = allocation_i + available_i$
- **Sequential Ordering:** A system is in a safe state if there exists a sequential ordering of all processes, denoted as $\langle P1, P2, \dots, Pn \rangle$.
- Iteratively do this four steps until any of need > work

STEP-2:(Cont.)

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2	5	3	2
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Sequential ordering
<P1,

STEP-2:(Cont.)

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2	5	3	2
P2	3	0	2	9	0	2	6	0	0	7	4	3
P3	2	1	1	2	2	2	0	1	1			
P4	0	0	2	4	3	3	4	3	1			

Sequential ordering
<P1,P3,

STEP-2:(Cont.)

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2	5	3	2
P2	3	0	2	9	0	2	6	0	0	7	4	3
P3	2	1	1	2	2	2	0	1	1	7	4	5
P4	0	0	2	4	3	3	4	3	1			

Sequential ordering
<P1,P3,P4,

STEP-2:(Cont.)

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2	5	3	2
P2	3	0	2	9	0	2	6	0	0	7	4	3
P3	2	1	1	2	2	2	0	1	1	7	4	5
P4	0	0	2	4	3	3	4	3	1	10	4	7

Sequential ordering
<P1,P3,P4,P2,

COMPLETION OF STEP-2!

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	3	3	2
P1	2	0	0	3	2	2	1	2	2	5	3	2
P2	3	0	2	9	0	2	6	0	0	7	4	3
P3	2	1	1	2	2	2	0	1	1	7	4	5
P4	0	0	2	4	3	3	4	3	1	10	4	7
										10	5	7

Sequential ordering

<P1,P3,P4,P2,P0>

THUS THE SYSTEM IS IN SAFE STATE!!

PROBLEM-2

IS THE SYSTEM IN SAFE STATE OR NOT?

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3				1	1	2
P1	2	0	0	3	2	2						
P2	3	0	2	9	0	2						
P3	2	1	1	3	2	5						
P4	0	0	2	4	3	3						

STEP-1:finding the need.

$$\text{Need}_i = \text{Max}_i - \text{Allocation}_i$$

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	1	1	2
P1	2	0	0	3	2	2	1	2	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	3	2	5	1	1	4			
P4	0	0	2	4	3	3	4	3	1			

STEP-2: Finding the values for available;

	ALLOCATION			MAXIMUM			NEED			AVAILABLE		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3	1	1	2
P1	2	0	0	3	2	2	1	2	2			
P2	3	0	2	9	0	2	6	0	0			
P3	2	1	1	3	2	5	1	1	4			
P4	0	0	2	4	3	3	4	3	1			

ALGORITHM:

Let Work and Finish be vectors of length m and n, respectively.

Initialize Work = Available and Finish[i] = false for $i = 0, 1, \dots, n - 1$.

2. Find an index i such that both
 a. Finish[i] == false
 b. $Need_i \leq Work$,If no such i exists, go to step 4.

3. Work = Work + Allocation_i
 Finish[i] = true Go to step 2.

4. If Finish[i] == true for all i, then the system is in a safe state,
 If Finish[i] == false for all i, then the system is in unsafe state.

WE REACH 2.b, thus the system is unsafe

THANK YOU!